

PROJECT DOCUMENTATION

Title: Retrieval-Augmented Generation (RAG) System using LangChain

1. Introduction

This project implements a **Retrieval-Augmented Generation (RAG) system** that enables users to query information from uploaded documents using natural language. The system combines **document retrieval techniques** with **Large Language Models (LLMs)** to generate accurate, context-aware responses.

The application supports both **single-document** and **multi-document querying**, allowing users to upload files in various formats and interact with their contents through a conversational interface. The system is built using **LangChain**, **Groq LLM (LLaMA 3.3)**, and **Streamlit**.

2. Objectives

- To design a system for querying documents using natural language
- To implement Retrieval-Augmented Generation using LangChain
- To support both single and multiple document workflows
- To integrate efficient embeddings for semantic search
- To enable fast response generation using Groq LLM
- To build an interactive interface for document-based Q&A

3. Technologies and Libraries Used

3.1 Technologies

- Python: Core development language
- Streamlit: User interface
- LangChain: RAG pipeline orchestration
- Groq API: LLM inference (LLaMA 3.3 70B)
- FAISS: Vector similarity search

3.2 Libraries

- langchain_community.document_loaders: Document parsing
- langchain_text_splitters: Text chunking
- langchain_community.embeddings: Embedding generation
- langchain_community.vectorstores: Vector database (FAISS)
- langchain_core.prompts: Prompt templates

- langchain_core.output_parsers: Output formatting
- python-dotenv: Environment configuration
- tempfile: Temporary file handling

4. System Architecture

4.1 Overall Workflow

Document Upload → Document Loader → Text Splitter → Embedding Model → Vector Store (FAISS) → Retriever → Prompt Template → LLM (Groq) → Response Generation

4.2 Module Description

1. Document Loader (core/loader.py)

- Supports PDF, TXT, DOCX, and HTML files
- Converts documents into LangChain document objects

2. Text Splitter (core/splitter.py)

- Uses RecursiveCharacterTextSplitter
- Splits documents into manageable chunks
- Ensures overlap for better context retention

3. Embedding Model (core/embeddings.py)

- Uses HuggingFace model: sentence-transformers/all-MiniLM-L6-v2
- Converts text chunks into vector representations

4. Vector Store (core/vectorstore.py)

- Uses FAISS for efficient similarity search
- Stores embeddings and enables retrieval

5. LLM Integration (core/llm.py)

- Uses Groq's LLaMA 3.3 model
- Temperature set to 0 for deterministic responses

6. RAG Pipelines (pipelines/)

- **Single Document Pipeline**
 - Processes one document
 - Retrieves relevant chunks for a query
 - Generates answer strictly from context

- **Multi Document Pipeline**

- Combines multiple documents
- Performs retrieval across all documents
- Generates unified response

7. Prompt Design

- Restricts answers to retrieved context
- Returns “I don’t know” if information is unavailable

8. Streamlit Applications

- app_single.py: Interface for single document querying
- app_multi.py: Interface for multiple document querying
- Handles file upload, chat interface, and response display

5. Features

- Supports multiple file formats (PDF, DOCX, TXT, HTML)
- Single and multi-document query capability
- Semantic search using embeddings
- Fast retrieval using FAISS
- Context-aware answer generation using RAG
- Interactive chat interface
- Temporary file handling for uploads

6. Code Implementation

```
import streamlit as st
```

```
import tempfile
```

```
from pipelines.multi_pipeline import create_multi_rag
```

```
st.set_page_config(page_title="Multi Doc RAG", page_icon="📖")
```

```
st.title("📖 Multi Document RAG")
```

```
if "chain" not in st.session_state:
    st.session_state.chain = None

if "messages" not in st.session_state:
    st.session_state.messages = []

files = st.file_uploader(
    "Upload multiple files",
    type=["pdf", "docx", "txt", "html"],
    accept_multiple_files=True
)

if files:
    paths = []

    for file in files:
        with tempfile.NamedTemporaryFile(delete=False, suffix=file.name) as tmp:
            tmp.write(file.read())
            paths.append(tmp.name)

    with st.spinner("Processing documents..."):
        st.session_state.chain = create_multi_rag(paths)

    st.success("All documents ready!")

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])

query = st.chat_input("Ask across documents...")
```

```

if query and st.session_state.chain:
    st.session_state.messages.append({"role": "user", "content": query})

    with st.chat_message("assistant"):
        response = st.session_state.chain.invoke(query)
        st.write(response)

    st.session_state.messages.append({"role": "assistant", "content": response})

import streamlit as st
import tempfile

from pipelines.single_pipeline import create_single_rag

# Page config
st.set_page_config(page_title="Single Doc RAG", page_icon="📄 ")

# Clean light UI
st.markdown("""
<style>
body {
    background-color: #ffffff;
}
.stChatMessage {
    border-radius: 10px;
    padding: 8px;
}
</style>
""", unsafe_allow_html=True)

st.title("📄 Single Document RAG")

```

```
# Session state
```

```
if "chain" not in st.session_state:
```

```
    st.session_state.chain = None
```

```
if "messages" not in st.session_state:
```

```
    st.session_state.messages = []
```

```
# File upload
```

```
file = st.file_uploader("Upload file", type=["pdf", "docx", "txt", "html"])
```

```
if file:
```

```
    with tempfile.NamedTemporaryFile(delete=False, suffix=file.name) as tmp:
```

```
        tmp.write(file.read())
```

```
        path = tmp.name
```

```
    with st.spinner("Processing document..."):
```

```
        st.session_state.chain = create_single_rag(path)
```

```
    st.success("✅ Document ready! Ask your questions.")
```

```
# 💬 Display chat history (always runs first)
```

```
for msg in st.session_state.messages:
```

```
    with st.chat_message(msg["role"]):
```

```
        st.write(msg["content"])
```

```
# 💬 Chat input
```

```
query = st.chat_input("Ask something about the document...")
```

```
if query:
```

```
    if not st.session_state.chain:
```

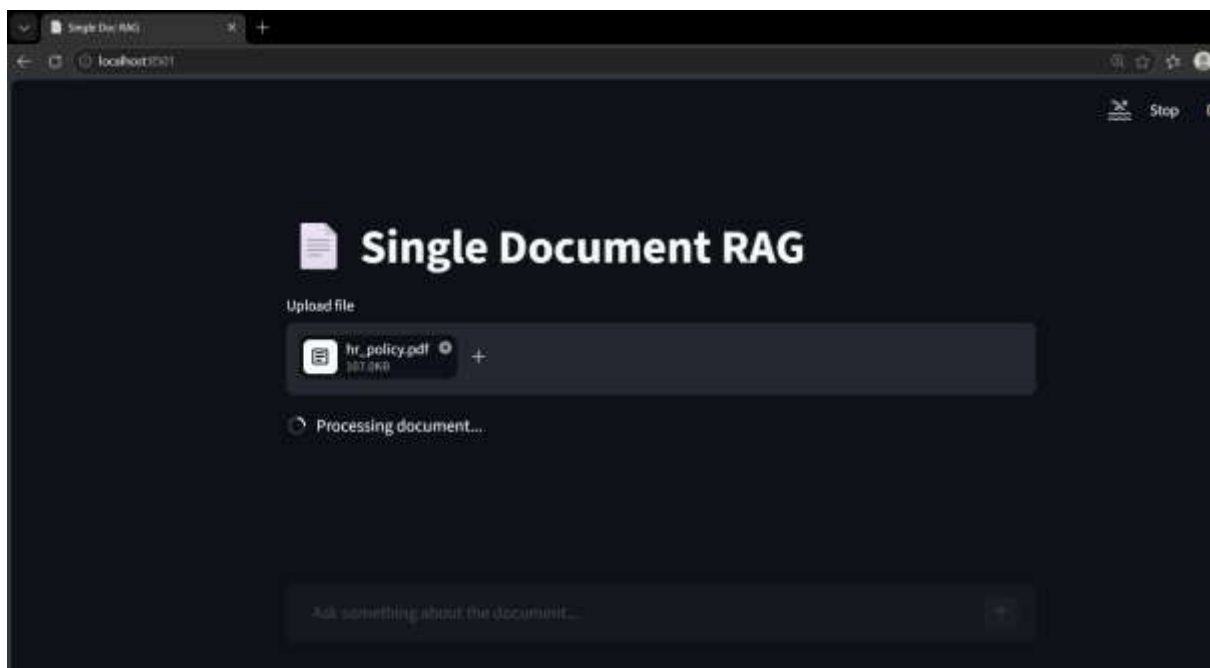
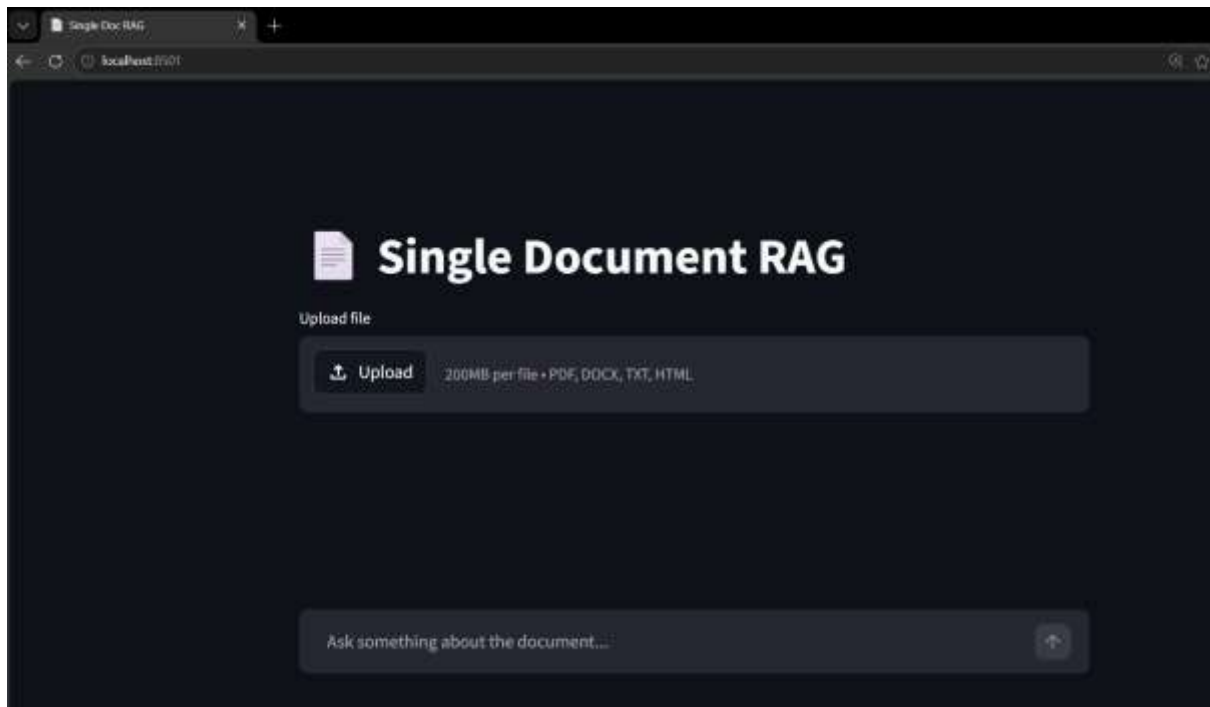
```
st.warning(" ⚠ Please upload a document first.")
else:
    # 1. Show user message immediately
    with st.chat_message("user"):
        st.write(query)

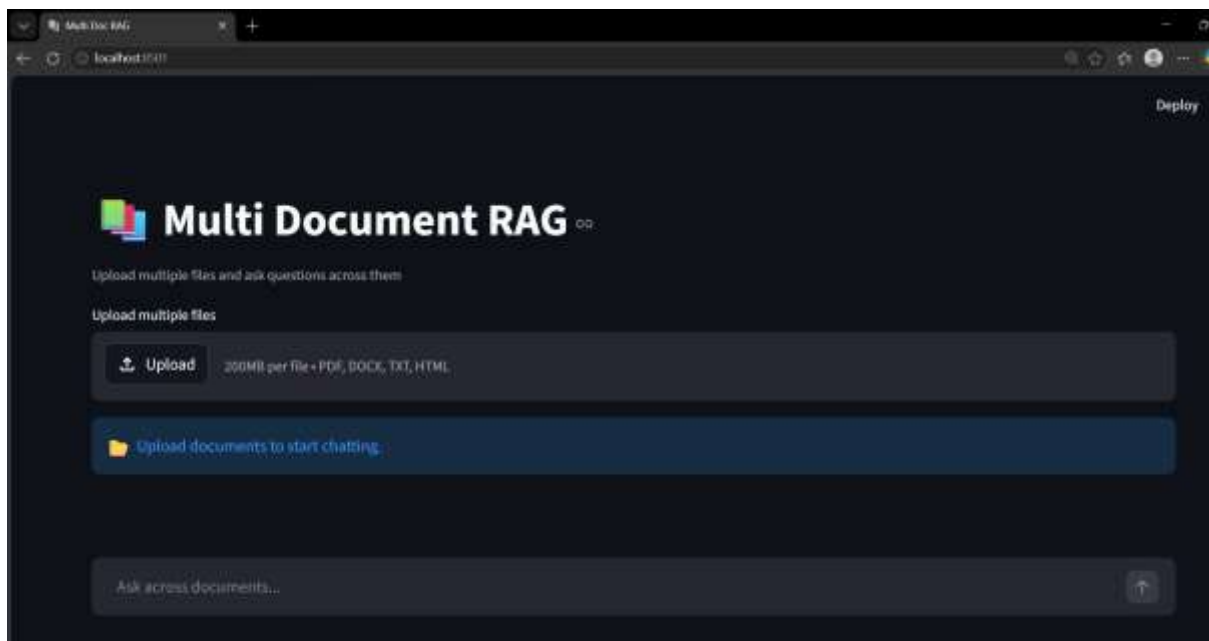
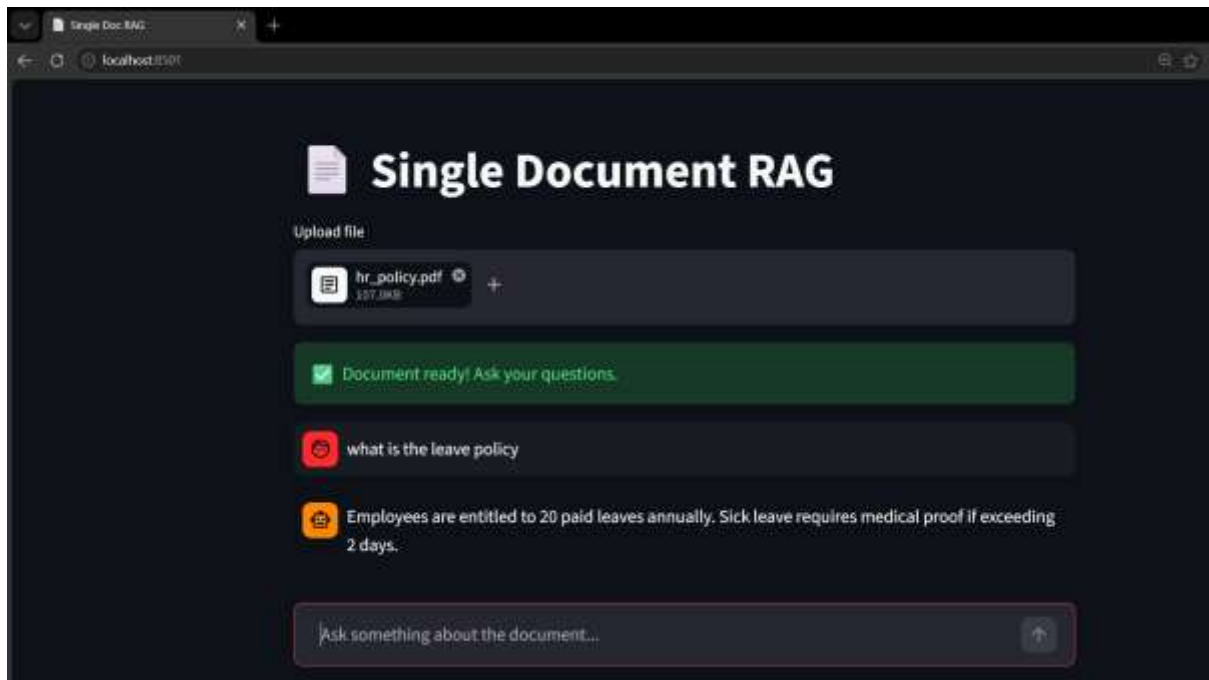
    # 2. Save user message
    st.session_state.messages.append({
        "role": "user",
        "content": query
    })

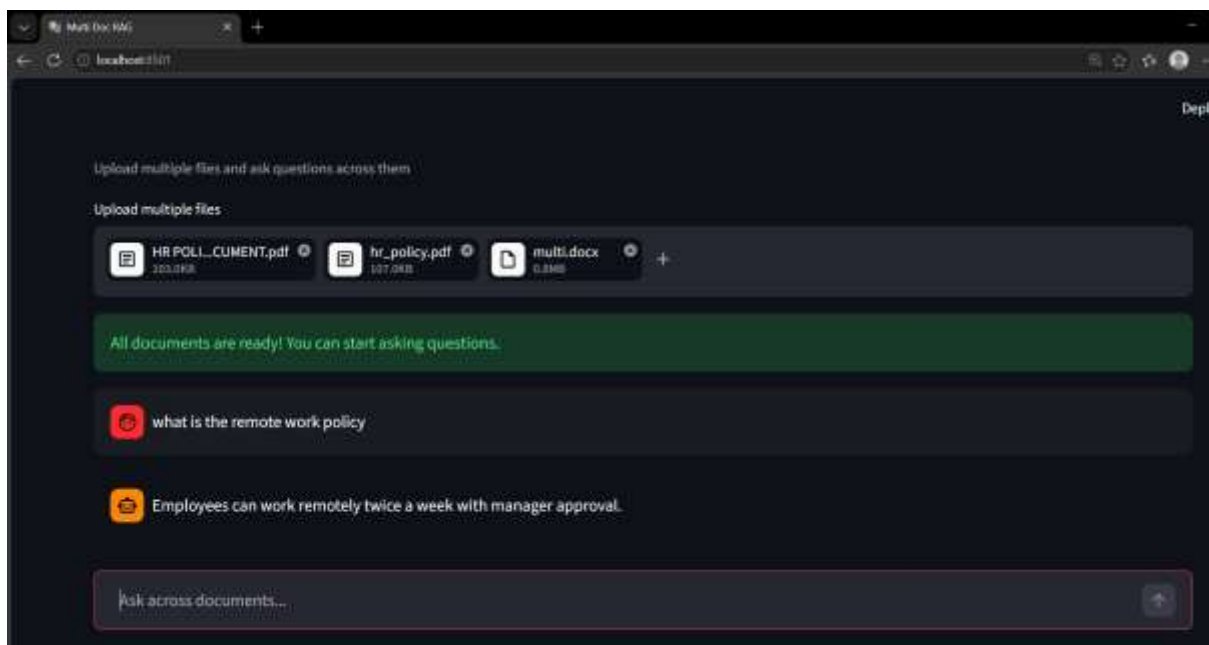
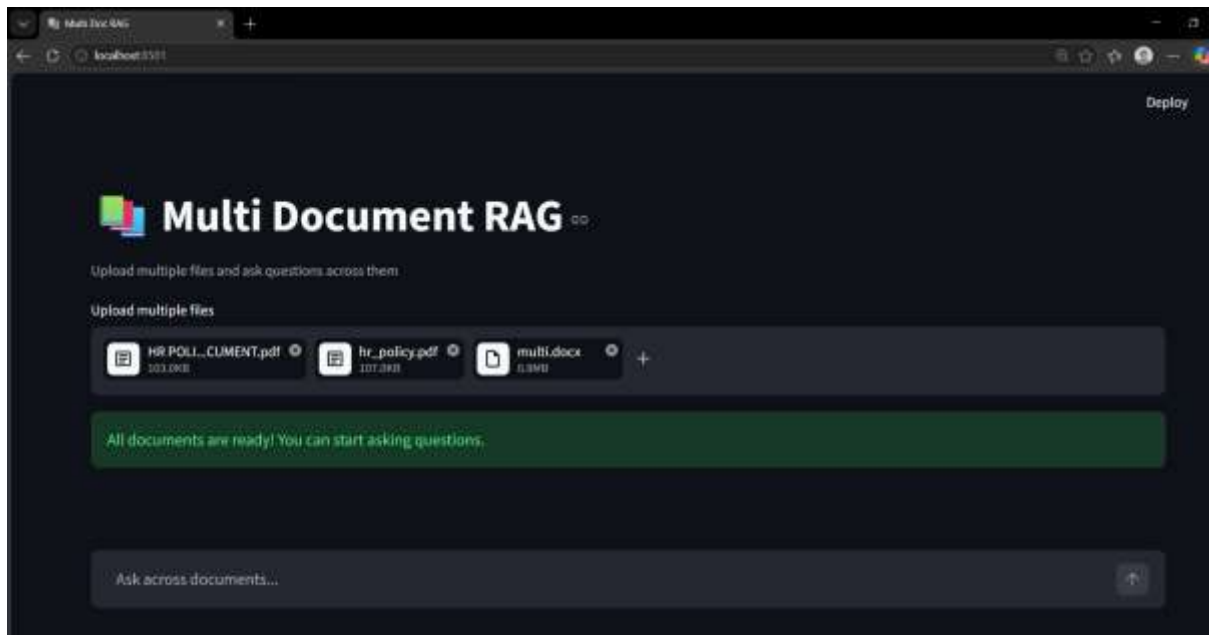
    # 3. Generate + show assistant response immediately
    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            response = st.session_state.chain.invoke(query)
            st.write(response)

    # 4. Save assistant response
    st.session_state.messages.append({
        "role": "assistant",
        "content": response
    })
```

7. Output







8. Conclusion

This project demonstrates the implementation of a Retrieval-Augmented Generation system using LangChain and Groq LLM. By combining document retrieval with language generation, the system enables accurate and context-aware responses. The modular design allows scalability and extension for real-world applications such as document analysis, knowledge systems, and enterprise search solutions.